Measurement Scope Matters: A Validation-First Benchmarking Framework for CPU and GPU AES Implementations

Marko Mihajlovic[1]

Abstract: This paper presents a validated and reproducible methodological framework for experimentally comparing cryptographic primitives and their implementations in a heterogeneous CPU/GPU environment. Starting from a DES/TDEA–AES retrospective, the study does not introduce a new cryptographic algorithm, but examines how implementation paths and measurement scope affect the interpretation of performance results. Particular emphasis is placed on a paired comparison between CPU AES-NI and CUDA AES-128 implementations, with a clear separation between isolated resident/kernel measurements and practical end-to-end pipelines that include host-device data transfers. The results show that GPU resident measurements can achieve high throughput for larger inputs, while the inclusion of transfer and synchronization costs changes the conclusion in favor of the AES-NI path in the tested environment. Reduced-keyspace and avalanche-effect demonstrations are used as controlled indicators of the limits of interpreting performance and security-related metrics. The results should not be interpreted as a general upper bound on GPU AES performance; they characterize the tested baseline CUDA implementation, hardware platform, and synchronous end-to-end pipeline.

Keywords: AES, DES, TDEA, AES-NI, CUDA, microbenchmarking, GPU, cryptographic performance, avalanche effect, reproducibility.

1 Introduction

The practical performance of cryptographic algorithms depends not only on their mathematical definitions but also on their implementation strategies, available hardware acceleration, and the methodology used to evaluate their execution costs. The same cryptographic primitive may exhibit substantially different performance characteristics when implemented in conventional CPU software, accelerated through dedicated processor instructions, or executed on a graphics processing unit (GPU). Consequently, comparisons based on a single throughput metric may be insufficient and potentially misleading unless the measurement scope and execution environment are explicitly defined.

The Data Encryption Standard (DES) and the Triple Data Encryption Algorithm (TDEA) remain relevant primarily from historical and methodological

[1] Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

perspectives. The DES specification and its associated standards have been withdrawn, while TDEA is no longer approved by the National Institute of Standards and Technology (NIST) for applying new cryptographic protection [1–4]. In contrast, the Advanced Encryption Standard (AES) remains a widely adopted standard for symmetric-key encryption [5]. Nevertheless, meaningful comparisons between cryptographic algorithms require consideration of their key lengths, block sizes, modes of operation, security properties, and implementation characteristics. Modes such as Cipher Block Chaining (CBC), Counter (CTR), and Galois/Counter Mode (GCM) introduce different computational requirements and security properties, further complicating direct performance comparisons [6, 7].

GPU acceleration of AES has been investigated in several studies, demonstrating the potential of massively parallel architectures to achieve high encryption throughput [8–12]. However, the interpretation of GPU performance results depends critically on which operations are included in the measurement. In particular, throughput obtained from an isolated GPU kernel operating on device-resident data does not necessarily represent the performance of a complete application-level encryption pipeline. Host-to-device transfers, device-to-host transfers, kernel launches, synchronization, and runtime overhead may substantially affect the observed end-to-end execution time. These distinctions are consistent with established CUDA performance measurement and optimization practices [13–15].

An additional challenge arises when GPU implementations are compared against CPU implementations that do not exploit available hardware acceleration. Modern x86 processors support dedicated AES instructions, commonly referred to as AES-NI, which significantly reduce the computational cost of AES operations compared with conventional software implementations [16]. Therefore, comparing a CUDA implementation exclusively against a software-based CPU implementation may overstate the practical advantage of GPU acceleration. A more informative evaluation should distinguish between conventional CPU software execution, hardware-accelerated CPU execution, GPU kernel execution, and the complete host-driven GPU pipeline.

The central research problem addressed in this paper is how to establish a methodologically consistent, validated, and reproducible framework for evaluating cryptographic primitives across heterogeneous computing environments. Rather than introducing a new encryption algorithm, cryptanalytic technique, or state-of-the-art CUDA implementation, the study focuses on the experimental conditions under which performance comparisons can be considered meaningful. Particular attention is given to functional correctness, controlled input generation, consistent workload definitions, explicit measurement boundaries, and the preservation of experimental evidence.

The proposed methodology follows a validation-first benchmarking approach

in which functional correctness is established before performance measurements are accepted. The experimental framework incorporates multiple execution profiles, including smoke, pilot, and study configurations, together with fixed random seeds, controlled input parameters, and paired comparisons. Measurements are organized into independent experimental sessions, while raw observations, aggregated summaries, validation results, and environment metadata are preserved to support reproducibility and subsequent analysis.

A central premise of the study is that performance claims must be interpreted within their specific measurement scopes. A CPU transform-only measurement, a GPU kernel-only measurement with device-resident data, and a complete GPU end-to-end measurement address different computational questions and should not be treated as interchangeable indicators of encryption performance. Accordingly, the experimental protocol distinguishes between conventional software AES, hardware-accelerated CPU AES-NI, CUDA resident-kernel execution, and host-driven CUDA end-to-end execution. This separation enables the computational benefits of GPU parallelism to be evaluated independently from the transfer and orchestration costs incurred by the complete processing pipeline.

The main methodological contribution is therefore the explicit separation of cryptographic correctness, computational throughput, and system-level execution costs within a reproducible experimental framework. The study further emphasizes three important interpretative distinctions: fixed-key GPU encryption throughput is not equivalent to exhaustive key-search performance; resident GPU throughput does not directly represent application-level throughput; and the avalanche effect, although useful for characterizing diffusion, does not constitute sufficient evidence of overall cryptographic security [5, 17]. By maintaining these distinctions, the proposed framework aims to provide a more reliable basis for evaluating heterogeneous implementations of symmetric cryptographic primitives and interpreting the practical significance of CPU–GPU performance differences.

2 Related Work and Research Gap

2.1 GPU Acceleration of AES

The acceleration of the Advanced Encryption Standard (AES) on graphics processing units has been investigated for nearly two decades. Early studies established the potential of GPU architectures to exploit the data-level parallelism available in block-cipher operations [8, 9]. In particular, the Electronic Codebook (ECB) and Counter (CTR) modes offer independent block-level computation, making them suitable for massively parallel execution. By contrast, Cipher Block Chaining (CBC) encryption introduces dependencies between consecutive ciphertext blocks, limiting direct parallelization within a single message. CBC decryp-

tion, however, permits parallel block-cipher operations when the corresponding ciphertext blocks are available [6].

Manavski [8] demonstrated an early CUDA-compatible implementation of AES, contributing to the exploration of programmable GPUs as cryptographic accelerators. Harrison and Waldron [9] investigated AES encryption and decryption on GPU architectures, providing additional evidence of the applicability of graphics hardware to symmetric cryptography.

Iwai et al. [10] subsequently examined the effects of thread granularity and memory placement on CUDA AES performance. Their experiments on an NVIDIA GeForce GTX 285 demonstrated that assigning 16 bytes per thread achieved approximately 35 Gbit/s. Importantly, the authors also investigated overlapping computation and data transfers, reporting approximately 22.5 Gbit/s when transfer time was included. This finding demonstrates that host–device communication had already been recognized as an important component of GPU cryptographic performance and that transfer-aware optimization predates the present study.

Wang and Chu [11] investigated GPU-accelerated AES encryption and decryption using ECB mode, lookup-table-based transformations, and one AES state per thread. Their work further illustrated how memory organization, scheduling granularity, and algorithm representation influence the achievable performance of GPU implementations.

More recent research has emphasized increasingly specialized kernel optimization techniques. Tezcan [12] introduced shared-memory optimizations designed to eliminate bank conflicts in table-based AES implementations. The reported AES-128 encryption throughput reached 878.6 Gbit/s on an NVIDIA RTX 2070 Super GPU, demonstrating the substantial performance potential of carefully optimized CUDA kernels.

Subsequent bit-sliced AES implementations have further advanced GPU encryption throughput. A study published in 2024 [18] reported peak AES-CTR throughput of approximately 1623 Gbit/s on an RTX 3080 and 60 Gbit/s on an embedded Jetson Orin Nano GPU. These results illustrate the continuing importance of algorithm representation, parallel counter generation, and device-specific optimization.

However, such throughput figures must be interpreted in relation to the workload, hardware generation, implementation strategy, and measurement boundaries used in each study. They should not be treated as directly interchangeable with throughput measured for a complete host-driven encryption pipeline.

The present work does not seek to reproduce these optimized implementations or claim competitive state-of-the-art kernel throughput. Instead, it uses

a functionally validated CUDA baseline and a lightweight optimization candidate to investigate how the definition of the measured execution path influences conclusions about CPU–GPU performance.

2.2 CPU Acceleration and Fair Heterogeneous Comparisons

A fundamental challenge in evaluating GPU-accelerated cryptography is the selection of an appropriate CPU reference. Conventional software implementations and hardware-accelerated CPU implementations may differ substantially in performance. Modern x86 processors provide dedicated AES instructions, commonly known as AES-NI, which accelerate important AES transformations in hardware [16].

Consequently, comparisons between optimized GPU kernels and conventional software-based CPU implementations may exaggerate the practical benefits of GPU acceleration. A meaningful heterogeneous comparison should include the hardware-accelerated CPU path and clearly identify whether the reported CPU implementation uses a single processing thread, multiple threads, or additional vector instruction extensions.

This concern extends beyond cryptography. Lee et al. [19] critically evaluated claims of exceptionally large GPU speedups and demonstrated that the observed performance advantage can decrease substantially when both architectures receive appropriate optimizations. Their study reported an average GPU advantage of approximately 2.5 times across the evaluated throughput-oriented kernels, rather than the much larger speedup factors sometimes reported in less balanced comparisons.

The relevance of this finding to cryptographic benchmarking is methodological rather than algorithm-specific. It suggests that GPU acceleration should not be evaluated against an artificially weak CPU baseline or without accounting for differences in implementation quality.

For this reason, the present study distinguishes conventional CPU software AES from hardware-accelerated CPU AES-NI. The AES-NI path serves as the primary CPU reference in the paired CPU–GPU comparison, while the software implementation provides an additional reference for evaluating the influence of processor-level acceleration.

Nevertheless, CPU implementation characteristics remain relevant. Library selection, instruction dispatch, thread count, mode-specific processing, and application-level invocation overhead may influence the measured throughput. Therefore, conclusions obtained using a specific CPU library and execution configuration must not be interpreted as universal limits of CPU AES performance.

2.3 Measurement Scope and Data-Movement Overhead

One of the most consequential methodological issues in heterogeneous performance evaluation is the boundary of the timed operation. GPU applications typically involve more than device-side computation. Depending on the workload and implementation, execution may include input preparation, host-memory staging, host-to-device (H2D) transfer, kernel launch, device execution, synchronization, and device-to-host (D2H) transfer.

Gregg and Hazelwood [20] demonstrated the importance of data placement and movement in CPU–GPU comparisons. Their work established that accelerator performance cannot be interpreted independently of where application data reside and which transfers are necessary to complete the evaluated workload.

This distinction is particularly important for cryptographic processing because the input and output buffers may originate from, and ultimately return to, host memory. In such scenarios, the throughput of an isolated device-resident encryption kernel is not necessarily representative of the application-visible encryption throughput.

For a sequential, non-overlapped GPU execution path, total processing time can be represented conceptually as

$$T_{\mathrm{GPU,E2E}} = T_{\mathrm{setup}} + T_{\mathrm{H2D}} + T_{\mathrm{kernel}} + T_{\mathrm{D2H}} + T_{\mathrm{sync}} + T_{\mathrm{other}}, \tag{1}$$

where the individual terms represent setup costs, data transfers, device computation, synchronization, and other implementation-dependent overheads. The exact inclusion of these components must be documented for each measurement.

By contrast, a resident-kernel measurement isolates the device execution component after input data have already been placed in GPU memory. Such a measurement is useful for investigating kernel efficiency, but it answers a different question from the end-to-end experiment.

CUDA programming guidelines emphasize that minimizing unnecessary data movement and overlapping transfers with computation are important techniques for improving application performance [13–15]. The CuPy performance guidelines similarly emphasize appropriate GPU timing procedures and the effects of asynchronous execution [21, 22].

These considerations motivate a distinction between three experimental concepts: the computational throughput of the cryptographic transformation, the throughput of a GPU kernel operating on resident data, and the throughput of a complete host-driven processing pipeline.

The present study explicitly separates these measurement scopes rather than treating them as equivalent estimates of practical encryption performance.

2.4 Scientific Benchmarking and Reproducibility

Beyond implementation efficiency, the reliability of a performance comparison depends on experimental design and statistical interpretation.

Mytkowicz et al. [23] demonstrated that apparently minor changes in experimental conditions can introduce systematic measurement bias and alter the conclusions of computer-system performance evaluations. Their findings highlight the need to control factors such as execution order, environment configuration, and other sources of experimental variability.

Kalibera and Jones [24] examined rigorous benchmarking under practical measurement-time constraints. Their work emphasizes the importance of accounting for variability across repeated observations and across different levels of experimental execution.

Hoefler and Belli [25] addressed the scientific interpretation of parallel-computing performance results, emphasizing statistically sound experimental design, meaningful performance reporting, and the distinction between reproducible effects and measurement noise.

These studies provide a methodological foundation for the present benchmarking protocol. Rather than relying exclusively on isolated timing observations, the framework organizes measurements into documented experimental sessions and preserves the information required to evaluate their consistency.

Functional correctness is verified before accepting performance measurements. Controlled inputs and fixed random seeds are used to reduce unnecessary workload variation, while paired experimental configurations are intended to improve the comparability of CPU and GPU execution paths.

The framework also records raw measurements, aggregated results, validation outcomes, and execution-environment metadata. This separation of experimental evidence from derived summaries is intended to support subsequent auditing and independent reanalysis.

However, successful cryptographic test-vector validation establishes the correctness of the evaluated implementations under the tested conditions; it does not, by itself, establish the statistical validity of the benchmarking methodology. The latter requires additional evidence concerning measurement stability, sensitivity to experimental conditions, and consistency with independently obtained measurements.

Accordingly, the methodological contribution is evaluated through its explicit measurement definitions, repeatable execution protocol, preserved artifacts, and analysis of the observed performance differences.

2.5 Research Gap and Contributions

The reviewed literature reveals two complementary research directions.

The first focuses on accelerating AES through parallel algorithm design, memory-placement strategies, table-based transformations, bit-slicing, and architecture-specific kernel optimizations [10–12, 18].

The second focuses on the methodological foundations of heterogeneous performance evaluation, including fair CPU–GPU comparisons, data-transfer costs, experimental variability, and scientifically defensible benchmarking practices [19, 20, 23–25].

These directions are closely related, but they address different aspects of performance evaluation. Kernel optimization studies establish what can be achieved under particular computational conditions, whereas measurement-methodology studies establish how performance results should be collected, compared, and interpreted.

Importantly, previous GPU AES research has already investigated data-transfer overhead and overlapping computation with communication [10]. Therefore, the distinction between kernel-only and transfer-inclusive measurements is not claimed as a new concept.

The research gap addressed by this paper is more specific: the need to bring cryptographic correctness validation, contemporary CPU AES-NI baselines, explicitly separated GPU measurement scopes, paired execution conditions, and preserved experimental artifacts into a unified and documented evaluation protocol.

Within this scope, the main contributions of the present study are summarized as follows:

1. **A validation-first cryptographic benchmarking protocol.** The study establishes correctness checks as a prerequisite for accepting timing measurements and defines controlled execution profiles, fixed workloads, and documented experimental configurations.

2. **An explicit separation of heterogeneous execution paths.** The protocol distinguishes conventional software AES, hardware-accelerated CPU AES-NI, GPU resident-kernel execution, and GPU end-to-end processing. This prevents computational and application-level throughput measurements from being treated as directly equivalent.

3. **A paired CPU AES-NI versus CUDA comparison.** The experimental design compares corresponding workloads under explicitly documented conditions, with AES-NI serving as the primary CPU reference and a software

implementation serving as an additional baseline.

4. **A reproducible measurement and reporting structure.** Raw timing observations, aggregated summaries, functional validation records, and execution metadata are retained to support transparent interpretation and subsequent independent analysis.

5. **An empirical assessment of measurement-scope dependence.** The study examines the extent to which observed CPU–GPU performance relationships depend on whether device computation or the complete host-driven pipeline is measured, while limiting its conclusions to the tested hardware, software stack, and execution configuration.

The scientific contribution is therefore not an assertion that one computational architecture is universally superior for AES encryption. Instead, it is an experimentally grounded assessment of how implementation choices, measurement boundaries, and reference baselines influence the interpretation of heterogeneous cryptographic performance.

This perspective is particularly relevant to systems in which encryption is one stage of a larger data-processing workflow, because the optimal processing location may depend on data residency, transfer costs, workload size, and the available CPU and GPU execution resources.

3 Methodology

This study adopts a validation-first experimental methodology for evaluating symmetric cryptographic primitives across CPU and GPU execution environments. The methodology is designed to distinguish functional correctness from performance, explicitly define the boundaries of each timed operation, and preserve the experimental evidence required for subsequent analysis.

The primary experimental design combines controlled workloads, paired CPU–GPU measurements, session-based execution, and hierarchical aggregation. Rather than treating encryption throughput as an implementation-independent property of a cryptographic algorithm, the study evaluates the performance of specific execution paths under documented hardware and software conditions.

3.1 Experimental Platform

The initial experimental platform consists of an Intel Core i5-10300H processor, 16 GB of system memory, and an NVIDIA GeForce GTX 1650 Ti GPU equipped with 4096 MiB of device memory. The system runs Microsoft Windows 10 and uses Python 3.12, PyCryptodome, CuPy, and the NVIDIA CUDA runtime.

PyCryptodome provides the CPU cryptographic implementations and reference operations used for functional validation [26]. Consequently, the CPU measurements characterize the performance of specific library execution paths rather than the maximum attainable performance of the underlying algorithms or processor architecture.

GPU implementations are executed through CUDA kernels integrated with CuPy. The CUDA execution environment provides device-memory management, kernel invocation, synchronization, and data-transfer functionality [13, 22].

The GPU operates under the Windows Display Driver Model (WDDM). This execution environment is relevant because GPU command scheduling, preemption, and synchronization interact with the operating-system driver stack [27, 28].

However, WDDM is treated as a potential source of system-level overhead and variability rather than an experimentally established cause of the measured CPU–GPU performance differences. Isolating its contribution would require controlled comparisons under different operating-system and driver configurations.

The performance results are therefore interpreted as measurements obtained on a particular hardware and software platform, not as general performance limits of AES or CUDA.

3.2 Validation-First Experimental Protocol

Functional correctness is treated as a prerequisite for performance evaluation. Before a benchmark configuration is accepted for timed execution, the corresponding cryptographic implementation must successfully complete the required validation procedures.

The validation process uses known-answer test vectors and functional consistency checks derived from the relevant cryptographic specifications [5–7].

Reference implementations provided by PyCryptodome are additionally used to verify selected CPU and GPU outputs [26].

A failed validation prevents the corresponding benchmark configuration from being accepted as a valid performance experiment. This design reduces the risk of reporting high throughput from an implementation that performs an incorrect or incomplete cryptographic transformation.

The experimental workflow distinguishes three execution profiles:

1. **Smoke profile:** A lightweight execution intended to verify basic functionality, benchmark configuration, validation procedures, and output-artifact generation.

2. **Pilot profile:** A preliminary measurement procedure used to examine tim-

ing behavior, execution stability, and the suitability of benchmark parameters before the final study.

3. **Study profile:** The controlled experimental configuration used to generate the measurements included in the principal performance analysis.

Only measurements obtained from the study profile are used to construct the principal performance tables.

The protocol preserves random seeds, execution arguments, workload parameters, repetition counts, validation outcomes, and environment metadata. Source-code snapshots or cryptographic hashes provide additional information for identifying the implementation associated with each experimental session.

This approach establishes traceability between the measured execution, the implementation under evaluation, and the resulting performance records.

It is important to distinguish implementation validation from methodological validation. Passing cryptographic test vectors demonstrates consistency with the expected cryptographic behavior under the tested conditions. It does not independently establish that the timing procedure is unbiased or sufficiently precise.

Consequently, the validity of the benchmarking methodology must also be assessed through measurement consistency, clearly defined timing boundaries, and sensitivity to experimental conditions.

3.3 Workload Definition and Paired Experimental Design

The experimental design uses controlled input parameters to improve the comparability of different execution paths. Corresponding CPU and GPU configurations are evaluated using matching workload definitions, including the cryptographic algorithm, mode of operation, data size, and relevant initialization parameters.

Fixed random seeds support deterministic input generation and facilitate the reconstruction of experimental conditions. However, identical seeds across executions do not eliminate environmental variability arising from scheduling, clock changes, memory behavior, or other system-level effects.

The primary heterogeneous comparison focuses on AES-128 execution using hardware-accelerated CPU AES-NI and CUDA implementations. Conventional CPU software execution is retained as an additional reference path.

The CUDA evaluation includes a baseline implementation and a lightweight optimization candidate. Both are subject to the same functional validation requirements and measurement conventions.

The paired design is intended to ensure that observed performance differences

reflect the selected execution paths rather than unintended differences in workload configuration.

Nevertheless, pairing does not imply that CPU and GPU implementations perform identical host-side operations. The timing boundaries of each execution path must therefore be specified before their results are interpreted.

The experimental design also distinguishes the characteristics of cryptographic modes. ECB and CTR permit independent block-level processing, whereas CBC encryption introduces sequential dependencies between ciphertext blocks [6].

Accordingly, performance comparisons are interpreted within individual cryptographic modes rather than treating throughput across different modes as interchangeable measurements.

3.4 Measurement Boundaries

A central element of the methodology is the explicit separation of execution paths according to their measured computational scope.

Four principal performance paths are distinguished:

1. **CPU software path:** Measures a conventional software-based cryptographic transformation through the selected CPU implementation.
2. **CPU AES-NI path:** Measures the corresponding cryptographic transformation through a hardware-accelerated CPU library path.
3. **GPU resident path:** Measures CUDA kernel execution on input data already resident in device memory, excluding the host–device transfers required to establish that resident state.
4. **GPU end-to-end path:** Measures the defined host-driven CUDA processing pipeline, including the selected host-to-device transfer, kernel execution, device-to-host transfer, and required synchronization operations.

The CPU measurements are classified as transform-only measurements because they focus on the cryptographic transformation call and do not necessarily include all operations that would occur in a complete application.

Similarly, the GPU resident measurement characterizes the computational performance of the CUDA kernel without including the transfer costs associated with moving input and output data between host and device memory.

The GPU end-to-end measurement represents a broader execution scope. In the initial implementation, the pipeline follows a sequential host-to-device (H2D), kernel execution, and device-to-host (D2H) sequence using pinned host memory.

Pinned host memory is relevant because page-locked buffers can support more efficient host–device transfer operations and, under suitable conditions, asynchronous transfers [13, 14].

However, using pinned memory does not imply that the complete pipeline automatically overlaps data transfers with kernel execution.

The present baseline is interpreted as a synchronous, non-overlapped processing pipeline unless a separate asynchronous execution configuration is explicitly introduced and measured.

Furthermore, the term end-to-end refers to the defined benchmark boundary. It does not automatically include every possible application-level cost, such as external I/O, application initialization, memory allocation, or host-side buffer preparation, unless those operations occur within the timed region.

This qualification is essential because an omitted host-side staging operation may produce a materially different performance estimate from a fully application-inclusive measurement.

The measurement boundaries are therefore treated as part of the experimental definition rather than as incidental implementation details.

3.5 Timing and Throughput Calculation

CPU execution time is measured using Python's monotonic high-resolution `time.perf_counter_ns()` interface. The timer provides elapsed wall-clock time with nanosecond representation, although the reported resolution does not imply nanosecond measurement accuracy.

GPU timing is handled separately because CUDA kernel launches and other GPU operations may execute asynchronously relative to the host. CUDA events and explicit synchronization are used where appropriate to obtain completed device-side execution measurements, following established CUDA and CuPy benchmarking recommendations [14, 21].

Host-observed and device-observed timing intervals are not assumed to be equivalent. A GPU event-based kernel measurement principally characterizes elapsed device execution, whereas a host-driven interval may include runtime dispatch and synchronization costs.

For a workload containing N processed bytes and measured elapsed time T in seconds, throughput is calculated as

$$R = \frac{N}{10^6 T}, \tag{2}$$

where R is expressed in decimal megabytes per second (MB/s).

This convention distinguishes decimal throughput units from binary memory-size units such as mebibytes (MiB).

For a fixed input size, throughput and elapsed time are inversely related. Consequently, a speedup factor can be expressed using the ratio of CPU and GPU execution times:

$$S_{\mathrm{CPU/GPU}} = \frac{T_{\mathrm{CPU}}}{T_{\mathrm{GPU}}}, \tag{3}$$

where both execution times must refer to the same workload and clearly identified measurement boundaries.

A value greater than one indicates that the GPU execution path completes the defined workload faster than the selected CPU reference. A value below one indicates that the CPU reference is faster.

The comparison does not automatically imply equivalent system-level functionality when one timing scope excludes transfers and the other does not. For this reason, resident and end-to-end speedup factors are reported and interpreted separately.

For small input sizes, elapsed time may provide a more informative measure than throughput because fixed launch and orchestration costs can dominate execution. Accordingly, latency-based reporting is preferable when interpreting very short operations.

3.6 Experimental Sessions and Statistical Aggregation

The initial performance study consists of five independent study sessions executed with documented random seeds and experimental arguments.

Each session includes repeated measurements for the corresponding benchmark configurations. Warm-up executions are excluded from the statistical summaries.

For a given configuration, the median of its valid timing observations is calculated independently within each study session.

Let $T_{s,r}$ represent the elapsed time recorded in session s and repetition r. The session-level median is defined as

$$\widetilde{T}_s = \mathrm{median}_r\left(T_{s,r}\right). \tag{4}$$

The principal aggregated execution-time statistic is the median of the five session-level medians:

$$\widetilde{T}_{\mathrm{study}} = \mathrm{median}_{s=1,\ldots,5}\left(\widetilde{T}_s\right). \tag{5}$$

This hierarchical aggregation preserves the distinction between repeated observations within a session and variation across independent sessions.

Raw observations from different sessions are not pooled into a single distribution for the calculation of the principal study median. Furthermore, no observations are excluded solely because they appear extreme.

For paired CPU–GPU comparisons, speedup factors are calculated from corresponding session-level timing medians:

$$S_s = \frac{\widetilde{T}_{\mathrm{CPU},s}}{\widetilde{T}_{\mathrm{GPU},s}}. \tag{6}$$

The aggregated speedup is then defined as

$$\widetilde{S}_{\mathrm{study}} = \mathrm{median}_{s=1,\ldots,5}(S_s). \tag{7}$$

This approach is distinct from calculating the ratio of independently aggregated CPU and GPU medians. Because the median is a nonlinear statistic, these two calculations need not produce identical results.

The session-level formulation preserves the intended pairing between CPU and GPU measurements.

Measurement stability is evaluated descriptively using variability across session-level summaries. Standard deviation and range are treated as descriptive indicators rather than formal confidence intervals.

The use of five sessions provides an initial assessment of experimental stability but does not support strong statistical generalization to different platforms, workloads, or execution environments.

These limitations are consistent with the broader literature on rigorous computer-system benchmarking, which emphasizes the importance of experimental variability, controlled repetition, and statistically defensible interpretation [23–25].

3.7 Experimental Artifacts and Traceability

The benchmarking framework preserves multiple categories of experimental artifacts to support traceability and independent analysis.

These artifacts include raw timing records, aggregated performance summaries, functional validation reports, execution metadata, accompanying reports, and source-code snapshots or cryptographic hashes.

Raw records preserve individual observations rather than only the final aggregated measurements. Summary records document the derived statistics used

in the performance analysis.

Validation records establish whether the measured implementations satisfied the required functional checks. Metadata records associate the observations with experimental parameters and the corresponding execution environment.

Source-code identification through snapshots or hash values helps distinguish results produced by different implementation revisions.

Together, these artifacts establish a documented relationship between experimental inputs, executable implementations, measurement observations, and reported conclusions.

The availability of such evidence improves auditability and supports subsequent reanalysis. Nevertheless, the preservation of artifacts alone does not guarantee reproducibility across different hardware or software environments.

Independent reproduction also requires sufficiently precise descriptions of dependencies, benchmark invocation procedures, synchronization behavior, and platform characteristics.

3.8 Methodological Scope and Limitations

The methodology is designed to support controlled interpretation of CPU–GPU cryptographic performance rather than establish universal rankings between computing architectures.

Several limitations must be considered.

First, the initial experiments are conducted on a single consumer-grade computing platform. Hardware characteristics, operating-system behavior, driver configuration, and runtime implementation may therefore influence the measured results.

Second, the CPU reference is derived from a particular cryptographic library and execution configuration. Although AES-NI provides an appropriate hardware-accelerated reference, the results do not necessarily represent the performance of independently optimized native or multithreaded CPU implementations.

Third, the CUDA baseline and lightweight optimization candidate are not intended to represent the highest attainable AES throughput on the tested GPU. Comparisons with highly optimized GPU implementations must therefore account for differences in kernel design and implementation maturity.

Fourth, resident and end-to-end GPU measurements describe distinct execution scopes. Their throughput values should not be interpreted as competing estimates of the same computational operation.

Fifth, the initial session count and descriptive statistical analysis limit the strength of general conclusions regarding platform-independent performance be-

havior.

Finally, the current synchronous pipeline does not establish the performance attainable through asynchronous transfers, concurrent execution, or optimized multi-stream processing.

These limitations motivate additional experiments involving independent CPU references, pipeline decomposition, asynchronous execution, denser workload-size sweeps, and evaluation on a second GPU platform.

The intended purpose of these extensions is not to demonstrate a predetermined architectural advantage, but to determine whether the observed performance relationships remain stable when implementation and execution conditions change.

4 Results

This section presents the experimental results obtained using the validation-first benchmarking methodology described in Section 3. The analysis begins with retrospective CPU comparisons of symmetric encryption algorithms, followed by authenticated encryption measurements and the principal paired CPU–GPU evaluation.

The main objective is not to identify a universally superior cryptographic implementation, but to examine how algorithm selection, hardware acceleration, and measurement scope influence the interpretation of observed performance.

Unless otherwise stated, throughput values are expressed in decimal megabytes per second (MB/s), and the reported aggregate values represent the corresponding statistics obtained from five study sessions.

4.1 CPU Comparison of DES, TDEA, and AES

The initial experiments establish a historical and implementation-oriented reference for symmetric encryption performance. DES, three-key TDEA, and AES-128/192/256 were evaluated in Cipher Block Chaining (CBC) mode without padding.

DES and TDEA are included exclusively for retrospective comparison and are not presented as recommended algorithms for protecting new data. Their historical standardization and subsequent withdrawal or restriction provide context for the evolution of symmetric cryptographic implementations [1–4].

Functional validation employed known-answer CBC test vectors derived from standardized sources [2, 6]. The AES experiments additionally distinguished conventional software execution from the hardware-accelerated AES-NI library path.

Table 1 summarizes the measured throughput and corresponding acceleration factors.

Table 1
CPU performance of CBC encryption and AES-NI experiments. Throughput is expressed in decimal MB/s; aggregated study-session medians are reported.

Experiment	Algorithm / operation	Software	AES-NI	Speedup
CBC, 10 MiB enc.	DES	87.4	–	–
CBC, 10 MiB enc.	TDEA	32.6	–	–
CBC, 10 MiB enc.	AES-128	223.8	–	–
CBC, 10 MiB enc.	AES-192	198.9	–	–
CBC, 10 MiB enc.	AES-256	174.6	–	–
AES-NI, 10 MiB enc.	AES-128	225.1	531.6	2.37×
AES-NI, 10 MiB dec.	AES-128	223.7	557.2	2.47×
AES-NI, 10 MiB enc.	AES-192	198.5	504.3	2.53×
AES-NI, 10 MiB dec.	AES-192	195.5	517.8	2.65×
AES-NI, 10 MiB enc.	AES-256	174.9	474.0	2.71×
AES-NI, 10 MiB dec.	AES-256	171.4	480.4	2.80×

The results demonstrate substantial implementation-dependent differences in encryption throughput. Among the conventional software paths, AES variants outperform TDEA under the evaluated CBC workload. AES-128 achieves the highest throughput among the software AES configurations, followed by AES-192 and AES-256.

This ordering is consistent with the increasing number of AES rounds associated with longer keys. AES-128, AES-192, and AES-256 require 10, 12, and 14 rounds, respectively [5]. However, the measured throughput differences also depend on the library implementation and should not be attributed exclusively to round counts.

Hardware acceleration considerably changes the observed performance relationship. The AES-NI path achieves approximately 2.4–2.8 times the throughput of the corresponding software execution path in the evaluated configurations.

For example, AES-128 CBC encryption increases from 225.1 MB/s to 531.6 MB/s, while AES-256 CBC encryption increases from 174.9 MB/s to 474.0 MB/s.

The relative acceleration is somewhat larger for AES-256 than for AES-128 in the reported measurements. Nevertheless, these factors characterize the specific processor, cryptographic library, workload size, and measurement boundary used in the experiment. They should not be interpreted as universal AES-NI speedup factors.

Importantly, the observed performance differences do not represent com-

parative cryptographic security. They quantify the execution costs of particular algorithm and implementation configurations.

These findings also motivate the selection of AES-NI, rather than conventional software AES alone, as the primary CPU reference in the subsequent heterogeneous CPU–GPU evaluation.

4.2 AES-GCM Authenticated Encryption

The second group of experiments extends the CPU analysis to authenticated encryption using AES in Galois/Counter Mode (GCM).

Unlike CBC encryption, which provides confidentiality without integrated authentication, GCM combines encryption with authentication of the ciphertext and optional associated data [7].

The experiment evaluates AES-128-GCM and AES-256-GCM using a 96-bit nonce, a 128-bit authentication tag, and 16 bytes of fixed additional authenticated data (AAD).

Functional validation includes authentication-tag verification and rejection of modified ciphertext, authentication tags, or associated data.

These validation requirements distinguish the GCM workload from confidentiality-only encryption experiments and prevent direct interpretation of CBC and GCM throughput as equivalent cryptographic operations.

Table 2 presents the measured AES-GCM throughput on the selected CPU AES-NI execution path.

Table 2
Throughput of AES-GCM authenticated encryption and decryption on the CPU AES-NI path, expressed in decimal MB/s.

Input size	Algorithm	Encryption	Decryption
1 KiB	AES-128-GCM	70.6	38.1
1 KiB	AES-256-GCM	70.1	38.0
1 MiB	AES-128-GCM	1148.2	1131.4
1 MiB	AES-256-GCM	1059.3	1059.8
100 MiB	AES-128-GCM	1179.7	1189.3
100 MiB	AES-256-GCM	1100.4	1080.4

The results show a pronounced dependence on input size.

For 1 KiB messages, the reported throughput is substantially lower than for larger inputs. This behavior is consistent with the increased relative contribution of fixed per-operation costs, including invocation overhead and mode-specific processing, when the payload is small.

At 1 MiB and 100 MiB, AES-128-GCM achieves approximately 1.13–1.19 GB/s, while AES-256-GCM achieves approximately 1.06–1.10 GB/s.

The differences between encryption and decryption are relatively small for larger inputs. However, a more pronounced asymmetry is observed for 1 KiB messages, particularly in AES-128-GCM, where encryption reaches 70.6 MB/s and decryption reaches 38.1 MB/s.

The current measurements establish this asymmetry as an observation but do not isolate its cause. Additional profiling would be required to determine the contributions of authentication verification, invocation overhead, and library-specific execution behavior.

For small messages, latency expressed in microseconds per operation would provide a more direct characterization than throughput alone.

These results demonstrate the importance of workload size and cryptographic functionality when interpreting performance measurements. They also establish a contemporary authenticated-encryption reference that complements the subsequent AES-128 ECB/CTR CPU–GPU experiments.

4.3 CUDA Baseline and Paired CPU–GPU Comparison

The central experiment evaluates AES-128 execution across conventional CPU software, hardware-accelerated CPU AES-NI, and two CUDA implementation variants.

The CUDA baseline uses a readable byte-oriented AES kernel in which each CUDA thread processes one 16-byte AES block. The execution configuration employs 128 threads per CUDA block.

ECB and CTR modes were selected because they permit independent block-level execution. ECB is used exclusively as a controlled microbenchmark of independent block transformations, while CTR provides a parallelizable counter-based encryption mode defined by NIST SP 800-38A [6].

ECB is not presented as a recommended mode for protecting general application data, since independent encryption of identical plaintext blocks can reveal structural information.

The GPU implementations use CUDA execution functionality exposed through CuPy, including custom kernel invocation and CUDA event-based measurement facilities [13, 21, 22, 29].

The paired protocol uses corresponding plaintext data, encryption keys, and initial CTR counter values for each mode, workload size, and repetition.

In addition to the baseline CUDA implementation, a lightweight optimization candidate, designated CUDA v1, is evaluated under the same experimental

protocol.

The comparison distinguishes two GPU measurement scopes:

1. **Resident kernel-only (K):** The CUDA kernel operates on data already available in device memory. Host–device transfer costs are excluded from the measured kernel execution.

2. **End-to-end pipeline (E2E):** The measurement includes the defined host-driven sequence consisting of host-to-device data transfer, CUDA kernel execution, device-to-host transfer, and required synchronization.

The CPU execution paths provide corresponding transform-only references.

Table 3 presents the principal throughput results.

Table 3
Paired AES-128 CPU–GPU throughput comparison in decimal MB/s. K denotes resident kernel-only execution, whereas E2E denotes the defined host-driven GPU pipeline.

		CPU		CUDA baseline		CUDA v1	
Mode	Size	Soft.	AES-NI	K	E2E	K	E2E
ECB	1 MiB	275.7	1793.5	4901.1	1339.1	5149.3	1320.1
ECB	16 MiB	280.5	1904.8	3667.5	1080.0	3424.4	866.2
ECB	32 MiB	282.1	1909.9	4753.5	1202.0	4792.6	1214.5
CTR	1 MiB	257.4	1381.7	5017.1	1313.1	4853.9	1291.0
CTR	16 MiB	264.3	1446.5	3655.6	1103.1	3576.2	1074.0
CTR	32 MiB	266.3	1455.8	4800.6	1204.5	4877.7	1209.1

The results reveal a consistent distinction between resident GPU throughput and the throughput of the complete measured GPU pipeline.

In all reported configurations, resident CUDA execution achieves higher throughput than the corresponding CPU AES-NI reference.

However, this relationship does not persist when the measured GPU execution includes host–device transfers and synchronization.

For example, in CTR mode with a 1 MiB input, the baseline resident kernel reaches 5017.1 MB/s, compared with 1381.7 MB/s for CPU AES-NI. When the end-to-end GPU pipeline is measured, throughput decreases to 1313.1 MB/s, which is below the AES-NI result.

A similar distinction appears at 32 MiB. The baseline resident CTR throughput reaches 4800.6 MB/s, whereas the corresponding pipeline achieves 1204.5 MB/s. The CPU AES-NI path reaches 1455.8 MB/s for the same workload size.

Thus, the observed GPU computational advantage does not translate into an end-to-end advantage under the tested synchronous pipeline configuration.

This result must be interpreted within the measured execution boundaries. The resident kernel and the CPU transform-only path provide useful comparisons of computational performance, but they do not represent identical application-level processing costs.

The end-to-end GPU measurement includes additional operations required to move data through the accelerator. The comparison therefore illustrates the practical importance of data residency and processing-pipeline organization, consistent with the broader heterogeneous benchmarking literature [20].

Importantly, the observed relationship does not establish an intrinsic limitation of GPU AES encryption. Different transfer configurations, asynchronous execution, stronger CUDA implementations, or alternative platforms could change the measured end-to-end result.

4.4 Speedup Analysis and Measurement-Scope Dependence

To examine the relative execution costs more directly, GPU speedup factors were calculated using the corresponding CPU AES-NI execution path as the reference.

The speedup statistics were derived from paired session-level timing medians and subsequently aggregated across five study sessions, as defined in Section 3.6.

A speedup factor greater than one indicates shorter GPU execution time for the corresponding measured scope, whereas a value below one indicates shorter CPU AES-NI execution time.

Table 4 presents the resulting factors for resident kernel-only (I) and end-to-end pipeline (P) execution.

Table 4

GPU speedup relative to CPU AES-NI, calculated as the median of five paired session-level timing ratios. I denotes isolated resident execution and P denotes the end-to-end pipeline. Values greater than one indicate shorter GPU execution time.

Mode	Size	Baseline I	v1 I	Baseline P	v1 P
ECB	1 MiB	2.65	2.91	0.75	0.77
ECB	16 MiB	1.89	1.77	0.58	0.42
ECB	32 MiB	2.50	2.52	0.63	0.64
CTR	1 MiB	3.66	3.54	0.95	0.91
CTR	16 MiB	2.58	2.51	0.75	0.74
CTR	32 MiB	3.30	3.40	0.82	0.83

The resident measurements indicate substantial acceleration relative to the selected AES-NI CPU reference. Depending on the mode, workload size, and CUDA implementation, the reported resident speedup ranges from approximately 1.77 to 3.66.

However, all reported end-to-end pipeline speedup factors remain below one.

The closest result to parity occurs for the baseline CTR implementation at 1 MiB, where the speedup factor is 0.95. At 32 MiB, the CTR pipeline factors are approximately 0.82–0.83, while the ECB pipeline factors are approximately 0.63–0.64.

These findings demonstrate that the relative ranking of CPU and GPU execution changes when the measurement boundary changes.

A comparison restricted to isolated CUDA kernel execution would suggest a GPU advantage across the evaluated workloads. A transfer-inclusive comparison using the tested synchronous pipeline instead favors the AES-NI CPU reference.

The lightweight CUDA v1 optimization does not provide a consistent performance improvement.

For some resident configurations, the optimized candidate achieves moderately higher throughput than the baseline. In other configurations, the result decreases. The most pronounced difference occurs for ECB at 16 MiB, where the v1 end-to-end pipeline reports lower throughput than the baseline configuration.

Because the optimization is not consistently beneficial and the existing measurements do not isolate the causes of the observed variation, these differences should not be interpreted as evidence of a generally superior CUDA implementation.

The non-monotonic behavior observed around 16 MiB also requires further investigation. Potential contributing factors include execution-environment variability, device clock behavior, memory allocation, transfer characteristics, and synchronization overhead.

Without component-level timing and additional workload-size measurements, assigning the observed reduction to any particular mechanism would be speculative.

The central finding is therefore methodological: the performance relationship between CPU AES-NI and CUDA AES depends on the measured execution scope.

For the tested platform and implementation, high resident GPU throughput does not guarantee higher transfer-inclusive pipeline throughput.

This finding supports the need to report kernel-only and end-to-end measurements separately, particularly when GPU performance is used to motivate architectural decisions in heterogeneous systems.

4.5 Reduced Keyspace and Avalanche Demonstrations

Two additional experiments investigate conceptual aspects of cryptographic evaluation: exhaustive search over a deliberately reduced keyspace and the avalanche effect.

These experiments serve a demonstrative and retrospective purpose. They are not intended to provide a new cryptanalytic result or to establish the practical security of the evaluated algorithms.

4.5.1 Reduced-Keyspace Search

The reduced-keyspace experiment uses AES-128 in ECB mode, with 24 variable key bits and a conventional CPU software execution path.

The timed operation includes candidate-key construction, cipher-object initialization and key scheduling, encryption of two blocks, relevant allocation operations, ciphertext comparison, and termination when a matching candidate is found.

Consequently, the measured candidate-processing rate characterizes the complete implemented search loop rather than the isolated throughput of the AES encryption primitive.

The purpose is to illustrate the growth of exhaustive-search cost with the number of candidate keys, not to estimate the performance of optimized cryptanalytic hardware.

Table 5 summarizes the experimental scenarios.

Table 5
CPU exhaustive-search demonstration over a reduced 24-bit AES-128 keyspace.

Scenario	Candidates	Time (s)	Candidates/s
Start	1	0.0000088	113 636
Middle	8 388 609	54.38	154 263
End	16 777 216	109.01	153 903
Random	6 800 733	43.00	154 079

The complete traversal of the reduced keyspace requires approximately 109 seconds, corresponding to a processing rate of approximately 153.9 thousand candidate keys per second.

The middle, end, and random scenarios exhibit similar candidate-processing rates, while the single-candidate start scenario is dominated by the extremely short measurement interval and should not be treated as a stable throughput estimate.

Only the complete-search scenario provides a direct empirical basis for a simple linear extrapolation under the assumption of constant candidate-processing rate.

However, such extrapolation remains specific to the measured software execution path. The result does not represent the exhaustive-search performance of DES, TDEA, GPUs, FPGA-based accelerators, ASIC implementations, or specialized cryptanalytic systems.

Moreover, fixed-key encryption throughput is not equivalent to candidate-key testing throughput. Exhaustive search requires additional work, particularly the generation or initialization of a candidate key and evaluation of a matching condition.

The distinction is fundamental because the two workloads differ in computational structure and measurement scope.

The present experiment therefore illustrates the limitations of interpreting encryption throughput as an exhaustive-search rate.

More advanced cryptanalytic techniques, including meet-in-the-middle attacks against multiple encryption constructions, require separate computational and memory-cost models and are outside the experimental scope of this study.

4.5.2 Avalanche Effect

The avalanche experiment examines the change in ciphertext output produced by modifying one bit of the plaintext or one effective bit of the cryptographic key.

For each tested pair, the Hamming distance between the original and modified ciphertext blocks is calculated.

The concept is related to the classical principle of diffusion in cryptographic systems, where changes in input information should influence multiple output components [17].

The Hamming distance is normalized by the block size: 64 bits for DES and TDEA, and 128 bits for AES.

Table 6 summarizes the observed median and mean Hamming distances for plaintext-bit (P) and effective key-bit (K) modifications.

For DES and TDEA, the median Hamming distance is 32 out of 64 bits. For all AES variants, the median is 64 out of 128 bits.

Thus, the normalized median is 0.5 across all ten experimental groups.

The normalized mean values range from approximately 0.4990 to 0.5007, indicating that the observed output differences are concentrated around half of the corresponding ciphertext block size.

Table 6
Avalanche-effect measurements following a one-bit change in plaintext (P) or effective key (K). HD denotes Hamming distance.

Algorithm	Variation	Block	Median HD	Mean HD	Median norm.	Mean norm.
DES	P	64	32	32.02	0.50	0.5003
DES	K	64	32	31.94	0.50	0.4990
TDEA	P	64	32	31.97	0.50	0.4995
TDEA	K	64	32	32.02	0.50	0.5004
AES-128	P	128	64	63.99	0.50	0.4999
AES-128	K	128	64	63.99	0.50	0.4999
AES-192	P	128	64	64.03	0.50	0.5002
AES-192	K	128	64	64.09	0.50	0.5007
AES-256	P	128	64	63.98	0.50	0.4998
AES-256	K	128	64	64.00	0.50	0.5000

This behavior is consistent with the expected local diffusion properties of established block ciphers.

However, the result must not be interpreted as proof of cryptographic security or as evidence that the evaluated algorithms provide equivalent protection.

The avalanche experiment is not a complete evaluation of the Strict Avalanche Criterion (SAC), the Bit Independence Criterion (BIC), or resistance to differential, linear, or other forms of cryptanalysis.

Furthermore, normalized average Hamming distance alone is insufficient to characterize the full distribution of output changes or to distinguish securely designed ciphers from deliberately weakened constructions.

Accordingly, the avalanche results are treated as a descriptive demonstration rather than as a basis for ranking algorithmic security.

5 Discussion

The experimental results highlight the importance of measurement boundaries, hardware acceleration, and execution-pipeline organization when comparing symmetric cryptographic implementations across heterogeneous computing platforms.

The principal finding is that the relative performance of CPU AES-NI and GPU AES-128 changes substantially depending on whether the measurement isolates device-side computation or includes the host-driven processing pipeline.

This distinction is particularly important because a high GPU kernel throughput value does not necessarily indicate a corresponding advantage in the complete

execution path.

The following discussion examines the implications of these observations, their relationship to previous research, and the methodological limitations that must be considered before extending the findings to other systems.

5.1 Resident Throughput Versus End-to-End Performance

The paired CPU–GPU experiments demonstrate a clear difference between isolated GPU execution and transfer-inclusive processing.

For AES-128 in CTR mode with a 32 MiB input, the CUDA baseline achieves a resident throughput of 4800.6 MB/s, compared with 1455.8 MB/s for the CPU AES-NI reference.

However, when the defined GPU end-to-end pipeline is measured, throughput decreases to 1204.5 MB/s.

A similar relationship is observed in ECB mode, where the resident CUDA implementation outperforms the CPU AES-NI path, while the end-to-end pipeline remains slower under the evaluated conditions.

These results indicate that device-side encryption performance alone is insufficient to characterize the practical cost of the measured GPU processing path.

The distinction is consistent with the general observations of Gregg and Hazelwood [20], who emphasized the importance of data placement and movement when interpreting heterogeneous CPU–GPU performance comparisons.

From a methodological perspective, the results demonstrate that different measurement scopes can produce different architectural rankings even when the cryptographic transformation and workload definition remain unchanged.

A resident-kernel measurement estimates the execution rate of the cryptographic computation after the input has been placed in device memory. In contrast, a host-driven pipeline measurement includes additional operations required to transfer data to and from the accelerator.

Consequently, a resident throughput advantage does not automatically translate into an application-level advantage.

Nevertheless, the interpretation depends on the intended workload.

For an application whose data already reside on the GPU, kernel-only throughput may be directly relevant because host–device transfers are not necessarily required for each cryptographic operation.

Conversely, applications that receive plaintext in host memory and require the resulting ciphertext on the CPU must account for the necessary transfers.

Neither measurement scope is inherently more correct. Rather, each characterizes a different execution scenario.

The methodological requirement is to specify these scenarios explicitly and avoid interpreting measurements obtained under different boundaries as directly interchangeable.

5.2 The Role of CPU AES-NI Acceleration

The experiments also demonstrate the importance of selecting an appropriate CPU reference.

In the CBC experiments, AES-NI improves throughput by approximately 2.4–2.8 times compared with the corresponding conventional software execution path.

The hardware-accelerated CPU implementation therefore constitutes a substantially stronger reference than the software-only implementation.

This distinction has direct implications for heterogeneous performance comparisons.

When evaluated against conventional software AES, a GPU implementation may exhibit a large apparent advantage. However, the same GPU execution path may provide a considerably smaller advantage, or no advantage, when compared against hardware-accelerated CPU AES-NI.

This observation is consistent with the broader methodological arguments presented by Lee et al. [19], who demonstrated that CPU–GPU performance comparisons can be strongly influenced by the quality and optimization level of the respective implementations.

Accordingly, the present study treats AES-NI as the primary CPU reference in the paired heterogeneous comparison.

However, the reported AES-NI results characterize a particular single-threaded library execution path on the Intel Core i5-10300H processor.

They do not represent the maximum throughput attainable from the entire CPU, nor do they establish a performance bound for independently optimized native implementations.

Different CPU libraries, instruction-dispatch mechanisms, vector instruction extensions, and multithreaded execution strategies may produce different results.

Therefore, the observation that CPU AES-NI outperforms the evaluated synchronous GPU end-to-end pipeline must remain conditional on the specific CPU implementation and workload configuration.

An independent native implementation, such as an OpenSSL EVP benchmark, would help determine whether the observed relationship is sensitive to the CPU software stack.

Furthermore, evaluating multiple CPU thread counts would distinguish

single-core or single-thread performance from aggregate processor throughput.

5.3 Pipeline Overhead and the Limits of Synchronous Execution

The present study deliberately evaluates a synchronous, non-overlapped CUDA end-to-end processing path.

For this configuration, the measured execution time includes the defined sequence of host-to-device transfer, kernel execution, device-to-host transfer, and synchronization.

The difference between resident-kernel throughput and end-to-end throughput indicates that operations outside the isolated kernel contribute substantially to the measured pipeline cost.

However, the existing aggregate measurements do not independently identify the contribution of each component.

In particular, it is not possible to determine from the reported throughput values alone how much of the additional execution time originates from host-to-device transfer, device-to-host transfer, synchronization, host-memory staging, runtime dispatch, or other implementation-dependent operations.

The Windows Display Driver Model (WDDM) may influence command scheduling and host-driven execution behavior, but the present experiment does not isolate its contribution [27, 28].

It would therefore be inappropriate to attribute the measured performance gap directly to WDDM without additional experiments.

CUDA programming practices provide several mechanisms for reducing the impact of data movement and orchestration overhead, including pinned host memory, asynchronous memory transfers, concurrent streams, and overlapping computation with communication [13–15].

Previous AES-specific research has also examined overlapping transfer and computation [10].

Consequently, the synchronous pipeline evaluated here should be considered one implementation configuration rather than a performance limit of GPU-based encryption.

For an idealized sequential pipeline, total execution time may be approximated as

$$T_{\mathrm{sync}}(N) \approx L_{\mathrm{GPU}} + T_{\mathrm{H2D}}(N) + T_{\mathrm{kernel}}(N) + T_{\mathrm{D2H}}(N), \tag{8}$$

where N is the workload size and L_{GPU} represents fixed launch, synchronization, and other overheads not already included in the individual terms.

In a chunked asynchronous implementation, transfers and computation may overlap when supported by the GPU hardware, memory configuration, stream dependencies, and workload structure.

Under ideal steady-state conditions, the dominant per-chunk processing time may approach

$$T_{\text{chunk,steady}} \gtrsim \max\left(T_{\text{H2D,chunk}}, T_{\text{kernel,chunk}}, T_{\text{D2H,chunk}}\right), \tag{9}$$

provided that the three stages can proceed concurrently without resource contention.

This expression is an idealized lower bound rather than a guaranteed achievable execution time.

Practical performance also depends on pipeline startup and draining, available copy engines, PCIe bandwidth, buffer allocation, synchronization dependencies, and the number and size of chunks.

Overlapping transfers with computation may therefore reduce end-to-end execution time, but the magnitude of any improvement must be established experimentally.

The current results demonstrate the cost of the evaluated synchronous pipeline; they do not establish the performance attainable by an optimized asynchronous implementation.

5.4 Workload Size and the CPU–GPU Crossover Problem

The relationship between CPU and GPU execution is also influenced by workload size.

For relatively small inputs, fixed costs associated with GPU invocation, command submission, transfer initialization, and synchronization may constitute a substantial fraction of total execution time.

As workload size increases, these fixed costs may become less important relative to the cost of processing the input data.

This creates the possibility of a crossover point at which the GPU becomes faster than the selected CPU reference.

However, the existence of such a point depends on the effective CPU processing rate, GPU kernel throughput, transfer bandwidth, and pipeline organization.

A simplified CPU execution model can be expressed as

$$T_{\text{CPU}}(N) \approx L_{\text{CPU}} + \frac{N}{R_{\text{CPU}}}, \tag{10}$$

where L_{CPU} is the fixed CPU execution overhead and R_{CPU} is the effective processing rate.

For a sequential GPU pipeline, a corresponding first-order model is

$$T_{\mathrm{GPU}}(N) \approx L_{\mathrm{GPU}} + \frac{N}{B_{\mathrm{H2D}}} + \frac{N}{R_{\mathrm{kernel}}} + \frac{N}{B_{\mathrm{D2H}}}, \tag{11}$$

where B_{H2D} and B_{D2H} represent effective transfer bandwidths and R_{kernel} represents effective GPU processing throughput.

Additional host-side staging costs must be included if they contribute materially to the measured execution interval.

A crossover size N_{cross} can be defined by

$$T_{\mathrm{CPU}}(N_{\mathrm{cross}}) = T_{\mathrm{GPU}}(N_{\mathrm{cross}}). \tag{12}$$

However, a positive and practically attainable solution is not guaranteed.

If the GPU pipeline has both greater fixed overhead and a larger effective processing time per byte than the CPU reference, increasing the workload size alone will not reverse the performance relationship within this model.

Furthermore, real execution costs may deviate from a simple linear model due to memory allocation behavior, cache effects, clock variation, transfer characteristics, and runtime scheduling.

The initial experiments cover 1 MiB, 16 MiB, and 32 MiB workloads in the paired CPU–GPU evaluation.

Although these measurements demonstrate the importance of execution scope, they are insufficient to establish a general crossover threshold.

The non-monotonic throughput behavior observed at 16 MiB further indicates that additional measurements are necessary before a predictive model can be considered reliable.

A denser workload-size sweep, combined with independent pipeline-component measurements, would allow the proposed model to be fitted and tested against observed execution times.

This extension would move the analysis beyond descriptive throughput comparison toward quantitative prediction of CPU–GPU execution costs.

5.5 Cross-Platform Generalization and the NVIDIA T4

The present experimental evidence is limited to a consumer-grade NVIDIA GeForce GTX 1650 Ti operating under Windows 10 and WDDM.

This platform provides a relevant example of GPU acceleration in a conventional personal-computing environment, but its performance characteristics should not be generalized to server and cloud GPU configurations.

A planned extension of the study involves repeating the benchmarking protocol on an NVIDIA T4 GPU in a Linux-based cloud environment.

Both GPUs belong to the NVIDIA Turing architecture family and support CUDA compute capability 7.5. Nevertheless, they differ in device resources, memory bandwidth, power characteristics, and operating environment.

The cloud platform will also introduce a different host CPU, memory subsystem, PCIe topology, and potentially virtualized execution resources.

Consequently, differences between the two systems cannot be attributed exclusively to the GPU model or operating system.

A cross-platform evaluation should therefore preserve the same workload definitions, functional validation requirements, measurement boundaries, and statistical aggregation procedures while documenting the environment-specific parameters separately.

The primary objective would be to determine whether the observed distinction between resident and end-to-end throughput remains relevant under substantially different execution conditions.

If the T4 demonstrates the same resident-versus-end-to-end ordering, the finding would provide additional evidence that the distinction is important beyond the initial laptop platform.

Conversely, if the T4 achieves an end-to-end advantage over the selected CPU reference, the result would demonstrate the sensitivity of the performance relationship to platform characteristics and pipeline costs.

Neither outcome would invalidate the methodological contribution.

Instead, the comparison would help identify the conditions under which particular CPU–GPU performance conclusions are applicable.

Additional telemetry, including GPU clock behavior, power limits, measured transfer bandwidth, and execution variability, would improve the interpretation of platform-dependent differences.

However, the T4 experiment alone would not isolate WDDM overhead because several hardware and software variables change simultaneously.

A controlled operating-system or driver comparison on otherwise equivalent hardware would be required for stronger causal conclusions.

5.6 Implications for Authenticated Encryption

Although the principal CPU–GPU comparison focuses on AES-128 ECB and CTR workloads, practical cryptographic applications frequently require authenticated encryption.

AES-GCM combines counter-mode encryption with authentication based on the GHASH function [7].

The CPU AES-GCM measurements included in this study provide a reference for authenticated-encryption performance, but the current experimental framework does not evaluate a corresponding GPU AES-GCM implementation.

This limitation is important because GPU AES-GCM performance depends not only on the AES encryption kernel but also on the implementation of finite-field multiplication and authentication-tag generation.

In particular, GHASH involves arithmetic over the binary field $\mathrm{GF}(2^{128})$, where carryless multiplication is a relevant computational primitive.

Recent NVIDIA developments have introduced hardware support for carryless multiply-accumulate operations through the PTX instruction `clmad`, documented in CUDA 13.3 for Ampere and newer GPU architectures with compute capability 8.0 or higher.

This instruction provides opportunities for accelerating GHASH and related binary-field computations on supported hardware.

However, neither the GTX 1650 Ti nor the NVIDIA T4 supports this instruction, as both are Turing-generation GPUs with compute capability 7.5.

Therefore, these newer instruction-level capabilities cannot be assumed to improve AES-GCM performance on either of the two intended experimental platforms.

A GPU AES-GCM extension on the T4 would require an implementation compatible with Turing hardware, while investigating newer carryless-multiplication instructions would require an additional Ampere-or-newer platform.

Furthermore, faster GHASH computation does not automatically imply proportionally higher end-to-end AES-GCM throughput.

The full authenticated-encryption pipeline includes counter generation, AES computation, GHASH processing, tag generation or verification, and potentially host–device transfers.

Its performance must therefore be measured as a complete authenticated-encryption workload rather than inferred from isolated GHASH throughput.

The distinction between kernel-level and application-level measurement remains relevant, but the computational balance may differ from the ECB and CTR

implementations evaluated in the current study.

5.7 Reproducibility and Statistical Limitations

The study incorporates several mechanisms intended to improve experimental traceability and repeatability.

These include correctness validation before timing, controlled workloads, documented random seeds, multiple study sessions, preservation of raw observations, and explicit separation of measurement scopes.

The principal performance results are aggregated from five independent study sessions using session-level medians.

This approach avoids indiscriminately pooling all repeated observations and preserves the distinction between within-session and between-session variability.

However, the number of sessions remains limited.

The reported median values provide descriptive evidence of performance behavior on the tested platform but do not establish narrow statistical uncertainty bounds.

Consequently, relatively small differences between implementation variants should be interpreted cautiously.

This qualification is particularly relevant to the CUDA baseline versus CUDA v1 comparison, where some observed throughput differences are modest and may be influenced by experimental variability.

The literature on scientific benchmarking emphasizes the importance of controlled execution, appropriate treatment of variability, and transparent reporting of statistical uncertainty [23–25].

Future experiments should therefore extend the number of independent sessions, report session-level dispersion, and introduce confidence intervals where supported by the experimental design.

Additional controls could examine sensitivity to warm-up procedures, execution ordering, synchronization behavior, and environmental changes.

Independent reference tools would provide further evidence concerning the accuracy and consistency of the measurement procedure.

It is also necessary to distinguish cryptographic correctness from benchmarking validity.

Passing known-answer tests demonstrates that the evaluated implementation produces expected cryptographic outputs under the tested conditions.

It does not independently establish that the performance measurement procedure is free from systematic bias.

The present framework provides a documented foundation for reproducible evaluation, but broader methodological validation requires these additional controls and independent measurements.

5.8 Broader Implications and Scope of the Findings

The findings have implications for both cryptographic benchmarking and heterogeneous application design.

First, selecting a computational accelerator based exclusively on isolated kernel throughput can lead to conclusions that do not reflect the cost of the intended processing workflow.

Second, comparisons against conventional software CPU implementations may exaggerate the benefits of accelerator execution when dedicated CPU cryptographic instructions are available.

Third, the location of application data is an essential factor in determining whether GPU acceleration is advantageous.

When plaintext and ciphertext remain resident in device memory, a GPU kernel may provide substantial computational throughput. When both input and output reside in host memory, transfer and orchestration costs may change the relative advantage.

Fourth, the selection of an execution strategy should be considered jointly with workload size, transfer configuration, CPU reference implementation, and cryptographic mode.

These observations suggest that cryptographic performance should be reported as a multidimensional experimental result rather than a single implementation-independent throughput value.

At minimum, meaningful heterogeneous comparisons should identify the cryptographic workload, hardware configuration, CPU acceleration mechanism, GPU execution scope, timing boundaries, and statistical aggregation procedure.

The principal contribution of this study is therefore methodological.

The results demonstrate how explicit measurement boundaries and controlled CPU–GPU comparisons can prevent misleading interpretations of accelerator performance.

However, the available evidence remains specific to the tested platform, implementation configurations, and synchronous GPU pipeline.

The study does not establish that CPU AES-NI is universally faster than GPU encryption, nor does it claim that the evaluated CUDA kernels represent the highest achievable GPU performance.

Instead, it shows that the measured CPU–GPU performance relationship

depends on the experimental definition of the processing path.

Extending the analysis to asynchronous pipelines, independent CPU implementations, additional workload sizes, and a second GPU platform would provide a stronger empirical basis for determining the generality of this conclusion.

6 Conclusion

This paper presented a validation-first, reproducible methodological framework for benchmarking symmetric cryptographic primitives in heterogeneous CPU–GPU computing environments. The primary contribution lies not in the development of a new cryptographic algorithm or optimization technique, but in a structured experimental protocol that integrates functional validation, controlled workloads, explicitly defined measurement boundaries, paired performance comparisons, session-level aggregation, and preservation of experimental artifacts. The framework provides a systematic basis for interpreting the performance of specific cryptographic implementations under documented hardware and software conditions.

The experimental results demonstrate that the choice of measurement scope can substantially alter conclusions regarding heterogeneous cryptographic performance. Hardware-accelerated AES-NI execution achieved approximately 2.4–2.8 times the throughput of the corresponding conventional software AES implementations in the evaluated CBC configurations. In the principal AES-128 CPU–GPU comparison, the CUDA baseline achieved resident throughput of 4800.6 MB/s for a 32 MiB CTR workload, compared with 1455.8 MB/s for the CPU AES-NI reference. However, when host-to-device transfer, kernel execution, device-to-host transfer, and synchronization were included in the measured GPU pipeline, throughput decreased to 1204.5 MB/s. Thus, the isolated computational advantage of the GPU did not translate into an end-to-end advantage under the tested synchronous execution configuration.

The evaluation of a lightweight CUDA optimization candidate further demonstrated that improvements in isolated kernel execution do not necessarily produce corresponding improvements in transfer-inclusive performance. Memory behavior, data movement, synchronization, and runtime overhead may influence the overall result independently of kernel-level throughput. These observations reinforce the importance of reporting resident-kernel and end-to-end pipeline measurements separately, rather than treating them as interchangeable indicators of accelerator performance.

The retrospective DES/TDEA comparisons, reduced-keyspace experiment, and avalanche analysis provided complementary methodological examples while remaining outside the central heterogeneous-performance contribution. In par-

ticular, fixed-key encryption throughput must not be interpreted as exhaustive key-search throughput, and avalanche measurements alone do not establish overall cryptographic security. These distinctions illustrate the broader importance of matching experimental metrics to the specific properties or operations being evaluated.

The findings are limited to the evaluated cryptographic library paths, CPU and GPU implementations, hardware platform, workload sizes, and synchronous CUDA processing pipeline. They do not establish a general upper bound on GPU AES throughput or demonstrate that CPU AES-NI is universally faster than GPU-based encryption. Moreover, the current study does not isolate the individual contributions of data-transfer, synchronization, and operating-system overhead, and its five-session statistical design limits broader generalization.

Future work will prioritize component-level pipeline measurements, asynchronous data transfers using `cudaMemcpyAsync`, pinned-memory buffering, and multiple CUDA streams. Additional experiments will investigate a wider range of workload sizes, independent native CPU AES-NI references, and a second experimental platform based on an NVIDIA T4 GPU under Linux. These measurements will support the development and validation of a quantitative execution-cost model and the identification of possible CPU–GPU crossover points. Further extensions may examine optimized CUDA AES kernels, GPU AES-GCM, and architecture-specific profiling using NVIDIA performance-analysis tools.

Overall, the study demonstrates that meaningful cryptographic performance evaluation requires more than comparing isolated throughput values. Functional correctness, execution boundaries, hardware acceleration, data residency, statistical treatment, and reproducible experimental records must be considered together. The central result is therefore methodological: the apparent performance advantage of a cryptographic implementation depends not only on its computational capabilities, but also on what is measured and how the complete execution path is defined.

7 Acknowledgments

Placeholder acknowledgments. Replace with appropriate funding or support information, or remove the paragraph if no acknowledgments are needed.

References

[1] National Institute of Standards and Technology. Data encryption standard (des). Federal Information Processing Standards Publication 46-3, National Institute of Standards and Technology, 1999. URL `https://csrc.nist.gov/pubs/fips/46-3/final`.

[2] National Bureau of Standards. Des modes of operation. Federal Information Processing

Standards Publication 81, National Bureau of Standards, 1980. URL https://csrc.nist.gov/pubs/fips/81/final.

[3] Elaine Barker and Nicky Mouha. Recommendation for the triple data encryption algorithm (tdea) block cipher. NIST Special Publication 800-67 Rev. 2, National Institute of Standards and Technology, 2017.

[4] Elaine Barker and Allen Roginsky. Transitioning the use of cryptographic algorithms and key lengths. NIST Special Publication 800-131A Rev. 2, National Institute of Standards and Technology, 2019.

[5] National Institute of Standards and Technology. Advanced encryption standard (aes). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023.

[6] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001.

[7] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.

[8] Svetlin A. Manavski. Cuda compatible gpu as an efficient hardware accelerator for aes cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, 2007. doi: 10.1109/ICSPC.2007.4728256.

[9] Owen Harrison and John Waldron. Aes encryption and decryption on the gpu. In *GPU Gems 3*, chapter 36. Addison-Wesley Professional, 2007. URL https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-and-decryption-gpu.

[10] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of aes encryption on cuda gpu. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[11] Canhui Wang and Xiaowen Chu. Gpu accelerated aes algorithm, 2019. URL https://arxiv.org/abs/1902.05234.

[12] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[13] NVIDIA Corporation. Cuda c++ programming guide, n.d.. URL https://docs.nvidia.com/cuda/cuda-c-programming-guide/. Accessed 2026.

[14] NVIDIA Corporation. Cuda c++ best practices guide, n.d.. URL https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/. Accessed 2026.

[15] Mark Harris. How to overlap data transfers in cuda c/c++. NVIDIA Technical Blog, 2012. URL https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/.

[16] Intel Corporation. Intel advanced encryption standard instructions (aes-ni), 2012. URL https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html.

[17] Claude E. Shannon. Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4):656–715, 1949. doi: 10.1002/j.1538-7305.1949.tb00928.x.

[18] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of aes-ctr and aes-ecb bit-sliced implementation on gpus. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[19] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100x gpu vs. cpu myth: An evaluation of throughput computing on cpu and gpu. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[20] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate cpu vs. gpu performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[21] CuPy. Performance best practices. `https://docs.cupy.dev/en/stable/user_guide/performance.html`, n.d.. Accessed 2026.

[22] CuPy. cupy.rawkernel. `https://docs.cupy.dev/en/stable/reference/generated/cupy.RawKernel.html`, n.d.. Accessed 2026.

[23] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[24] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, 2013. doi: 10.1145/2464157.2464160.

[25] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[26] PyCryptodome. Pycryptodome documentation, n.d. URL `https://pycryptodome.readthedocs.io/`. Accessed 2026.

[27] Microsoft. Gpu preemption. Microsoft Learn, n.d.. URL `https://learn.microsoft.com/en-us/windows-hardware/drivers/display/gpu-preemption`. Accessed 2026.

[28] Microsoft. Timeout detection and recovery (tdr). Microsoft Learn, n.d.. URL `https://learn.microsoft.com/en-us/windows-hardware/drivers/display/timeout-detection-and-recovery`. Accessed 2026.

[29] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. Cupy: A numpy-compatible library for nvidia gpu calculations. In *Proceedings of Workshop on Machine Learning Systems (LearningSys) in NeurIPS 2017*, 2017. URL `https://cupy.dev/`.

[30] NVIDIA Corporation. Cuda installation guide for microsoft windows, n.d.. URL `https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/`. Accessed 2026.

[31] Jean-Luc Watson, Charles Gouert, Michael B. Sullivan, and G. Edward Suh. Building faster cryptography with carryless multiplication in nvidia cuda 13.3. NVIDIA Technical Blog, July 2026. URL `https://developer.nvidia.com/blog/?p=119649`. Published July 15, 2026.

[32] NVIDIA Corporation. Parallel thread execution isa: Carryless multiply-add instruction. NVIDIA CUDA Documentation, 2026. URL `https://docs.nvidia.com/cuda/parallel-thread-execution/`. PTX instruction clmad, introduced in PTX ISA 9.3; requires sm_80 or higher.